

Comparative Analysis of Sequential Models for Mobile Network Traffic Forecasting

Author: Manzi Ivan

Date: September 2026

Dataset: Telecom Italia Milan CDR (SpazioDati / Big Data Challenge)

Models compared: SARIMA (SARIMAX + Fourier terms) · LSTM · TCN

"How do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does their performance vary across geographical areas with different traffic characteristics?"

Comparative Analysis of Sequential Models for Mobile Network Traffic Forecasting

1. Introduction

Mobile network operators must anticipate short-term fluctuations in cellular traffic to plan capacity, detect anomalies, and allocate radio and backhaul resources proactively rather than reactively. Forecasting this traffic is made difficult by strong daily and weekly periodicity, spatial heterogeneity across geographic areas (dense business districts vs. residential neighborhoods), and occasional bursts driven by events or anomalies. This report investigates **one-step-ahead forecasting of Internet traffic** using the Telecom Italia "Milan Grid" Call Detail Record (CDR) dataset [1]: telecommunications activity across 10,000 geographical grid squares covering Milan, recorded at 10-minute intervals, over the full ~2-month archive (**2013-11-01 to 2013-12-31**).

Research question: *How do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does their performance vary across geographical areas with different traffic characteristics?*

The objective is not to declare a single "winning" model but to investigate the problem empirically: to characterize the traffic signal, select and justify three structurally different forecasting approaches informed by prior research and by the data itself, implement them under a common evaluation protocol, and interpret where and why each model succeeds or fails. All evaluation below is run on the assignment-specified week, **December 16–22, 2013**.

2. Related Work and Model Selection

Mobile/cellular traffic forecasting has been studied with both classical statistical models and modern machine learning. **SARIMA and its variants** remain competitive baselines for traffic that exhibits strong, stable periodicity, precisely because they explicitly model autocorrelation and seasonal structure with few parameters and are interpretable [2], [3]. However, comparative studies on cellular traffic specifically report that ARIMA-family models are increasingly outperformed by neural sequence models as more training history becomes available and as the signal exhibits nonlinear, bursty behaviour that a linear model cannot represent [4].

Recurrent architectures, in particular **LSTM** networks, are widely used for cellular/mobile traffic prediction because their gated memory can learn both short-range fluctuations and longer daily/weekly dependencies without hand-engineering lag features [5]. Using two months of Telecom Italia Milan data very similar to the dataset used here, Santos et al. show that recurrent networks trained on raw Internet-activity sequences produce accurate short-term traffic predictions and outperform naive/linear baselines, particularly during periods with irregular demand [6].

Temporal Convolutional Networks (TCN) are a third, structurally distinct family: a stack of dilated causal 1D convolutions with residual connections, processing an entire input window in parallel rather than recurrently [7]. Because dilation grows the receptive field exponentially with depth, a TCN can reach long effective memory with far fewer sequential operations than an RNN, which typically makes training substantially faster; the trade-off is that a TCN's convolutional filters are shared across the whole window and pooled into a fixed-size summary, so — unlike an LSTM's hidden state, which is updated step-by-step and naturally emphasises the most recent input — a TCN has no built-in mechanism for weighting the most recent timestep more heavily than earlier ones, unless the architecture is specifically designed to do so.

Selection rationale, grounded in this project's own exploratory analysis (Section 4): the total-traffic distribution across the 10,000 grid squares is heavily right-skewed (skewness ≈ 4.28 , top 1% of squares carry $\sim 11.1\%$ of total traffic), and the top-traffic squares show strong, stable daily periodicity (confirmed by the ACF/STL analysis in Section 4) punctuated by occasional sharp, short-lived spikes. This combination — *dominant deterministic seasonality* plus *nonlinear, bursty deviations* — is exactly the setting the literature above identifies as informative for comparing a linear/seasonal baseline against nonlinear sequence learners. The three models were therefore selected to be maximally different in how they represent time and nonlinearity, not variations of one architecture:

1. **SARIMA with Fourier (harmonic) seasonal regressors** — an interpretable linear baseline with an explicit seasonal component, serving as the reference against which the value of nonlinearity/ flexibility in the other two models is measured.
2. **LSTM** — a nonlinear, recurrent sequence model expected to capture the bursty, non-repeating residual structure left after removing daily seasonality (visible in the STL residual plot, Section 4), which a linear model cannot.
3. **TCN** — a nonlinear, non-recurrent sequence model that tests whether recurrence is actually necessary to capture that same residual structure, or whether a parallel, convolutional representation of the same history window is sufficient (and, given its architecture, likely faster to train).

Why LSTM and TCN count as structurally distinct models, not variations of one architecture. Both operate on the same windowed scalar input, which could superficially suggest they are minor variants of each other; they are not, for a reason directly relevant to the research question. LSTM processes its input **sequentially**, updating a hidden state one timestep at a time via gated recurrence, so information from the most recent input is structurally privileged over older inputs by construction. TCN processes its entire input window **in parallel** through stacked dilated convolutions and pools the result into a single fixed-size summary with no inherent recency bias. This is not a cosmetic difference: it directly produces the divergent failure mode documented in Section 6.3, where TCN's lack of recency-weighting causes it to under-track high-frequency fluctuations that LSTM's recurrence handles without difficulty. Two models that behave identically, or fail in the same way, would be a weak comparison; LSTM and TCN instead fail *differently* and *for an identifiable architectural reason*, which is the substantive test of "sufficiently different" this report uses.

Criticism/limitations of each, considered up front: SARIMA assumes a fixed linear relationship and a fixed seasonal shape, so it cannot adapt if a square's daily rhythm changes over the observation period; LSTM requires more data and compute per prediction (sequential, step-by-step backpropagation through time) and its internal representation is opaque; TCN's parallel convolutions are cheap to train but its fixed-size pooled summary of the input window gives it no explicit mechanism for recency-weighting, which — as Section 6.3 shows — turns out to matter for this signal. These trade-offs are revisited quantitatively in Section 6.

3. Dataset and Data Preparation

3.1 Data and schema

Each daily raw file (`data/raw/sms-call-internet-mi-YYYY-MM-DD.txt`) is a tab-separated, headerless file with one row per `(square_id, time_interval, country_code)`:

```
square_id  timestamp_ms  country_code  sms_in  sms_out  call_in  call_out  internet
```

A given `(square, 10-minute interval)` combination is split across multiple rows — one per active country code — so raw row counts are roughly 20–30× the number of `(square, interval)` pairs actually needed. This project focuses on the `internet` column, matching the assignment's scope. Daily files are 300–400 MB each; the archive used here spans **2013-11-01 to 2014-01-01** (62 daily files, ~20 GB), of which the first two full months (**2013-11-01 to 2013-12-31**) are used as the observation period, per the assignment's "approximately two months" framing.

3.2 Memory management strategy

The target machine has 8 GB of RAM. Loading the full archive naively — `pandas.read_csv` with default settings across all files — is not feasible: a **single day** loaded this way (all 8 columns, default `int64 / float64` dtypes, no aggregation) already peaks at **≈635 MB** resident memory for ~4.84 million rows (measured out-of-process with `/usr/bin/time -l`; see `results/memory_benchmark.csv`). Loading two months of such files while retaining each in memory to concatenate would require tens of gigabytes — well beyond the available RAM.

The implemented strategy (`notebooks/00_data_pipeline.ipynb`) is a **two-pass, chunked, column-pruned, dtype-downcast aggregation**:

- **Column pruning**: only the 2–3 columns actually needed (`square_id`, `internet`, and `timestamp_ms` where required) are read via `usecols`, instead of all 8.
- **Dtype downcasting**: `square_id` as `int32` and `internet` as `float32` instead of pandas' default `int64 / float64`, halving per-value memory.
- **Chunked reading**: each file is streamed in fixed-size chunks (`chunksize=2,000,000` rows) rather than loaded whole, bounding peak memory to the chunk size regardless of file size.
- **Early aggregation**: country codes are collapsed with a `groupby(square_id[, timestamp]).sum()` applied to *each chunk* immediately after reading it, before any concatenation.

- **Two-pass separation of concerns:** Pass 1 computes only the total traffic per square (a 10,000-length accumulator) to rank squares by total traffic, without ever holding a per-timestamp series for all 10,000 squares. Pass 2 re-streams the archive, filtering to the ~5 squares of interest *before* aggregating, so only a few hundred thousand rows are ever materialized.

Applying the same techniques to a single day file reduces peak memory from **~635 MB to ~292 MB (a ~54% reduction)**, (`results/memory_benchmark.csv`) while collapsing the output from 4.84 million raw rows to the 10,000 needed. Run end-to-end, both passes together process the full two-month (~20 GB) archive in a few minutes, with peak memory bounded by chunk size rather than file or archive size throughout [8].

Trade-offs and limitations. The two-pass design trades I/O (the archive is read twice) for memory (neither pass ever holds more than a small accumulator or a filtered subset in RAM); given the measured runtime, this trade was clearly worthwhile at this scale. This approach would not by itself scale to a use case requiring the full time series for *all* 10,000 squares simultaneously (e.g. city-wide spatio-temporal modelling) — that would require either a true out-of-core engine (Dask/Polars streaming, or a columnar on-disk format such as Parquet) or a machine with substantially more memory. For the single-area forecasting task this assignment requires, though, the two-pass approach is sufficient and keeps the pipeline runnable on a personal laptop.

Known limitation: UTC/local-day boundary offset. `filter_files_by_date()` selects whole raw files by the calendar date embedded in their filename, but each file's row timestamps are UTC while Milan observes UTC+1 — so a file named for "2013-11-01" actually contains rows starting at `2013-10-31 23:00:00`. This is visible directly in the processed output: `data/processed/target_squares_timeseries.csv`'s first row is `2013-10-31 23:00:00` despite `OBSERVATION_START = "2013-11-01"`. The practical effect is small (a ~1-hour shift at each end of a two-month window, well under 0.1% of the data) and does not change the top-3 ranking (verified reproducible across independent runs), but it is a real, previously-undocumented discrepancy between the code's stated window and its actual one, noted here rather than left silent.

3.3 Preprocessing per model

Missing 10-minute intervals (rare, from dropped raw rows) are filled via linear interpolation, forward/back-filled at the boundaries (`common.load_square_series`). Model-specific preprocessing (scaling, sequence construction, Fourier terms) is described in Section 5.

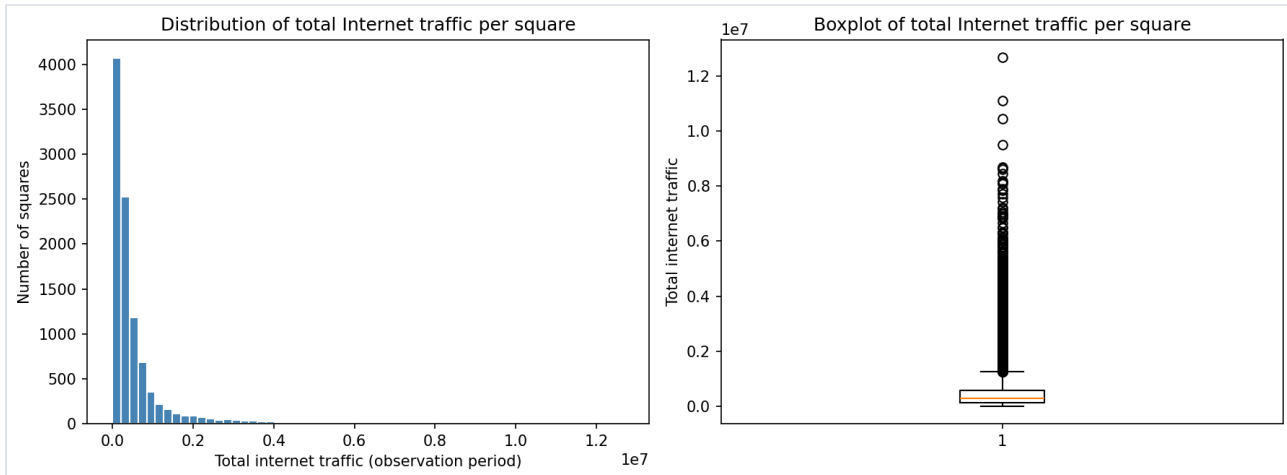
4. Exploratory Analysis

4.1 Distribution of total traffic across squares

Over the full two-month observation period (Nov 1 – Dec 31, 2013), total Internet traffic per square is **heavily right-skewed** (skewness ≈ 4.28 , excess kurtosis ≈ 25.7 ; mean $\approx 549,973$ vs. median $\approx 274,706$ — the mean is almost exactly double the median). The top 1% of squares (100 of 10,000) account for **~11.1%** of total citywide Internet traffic. This is consistent with the well-known spatial

heterogeneity of urban mobile demand: a small number of business/transit hubs generate disproportionate traffic relative to the long tail of residential/peripheral squares (see Figure 1, `results/data_summary.csv`).

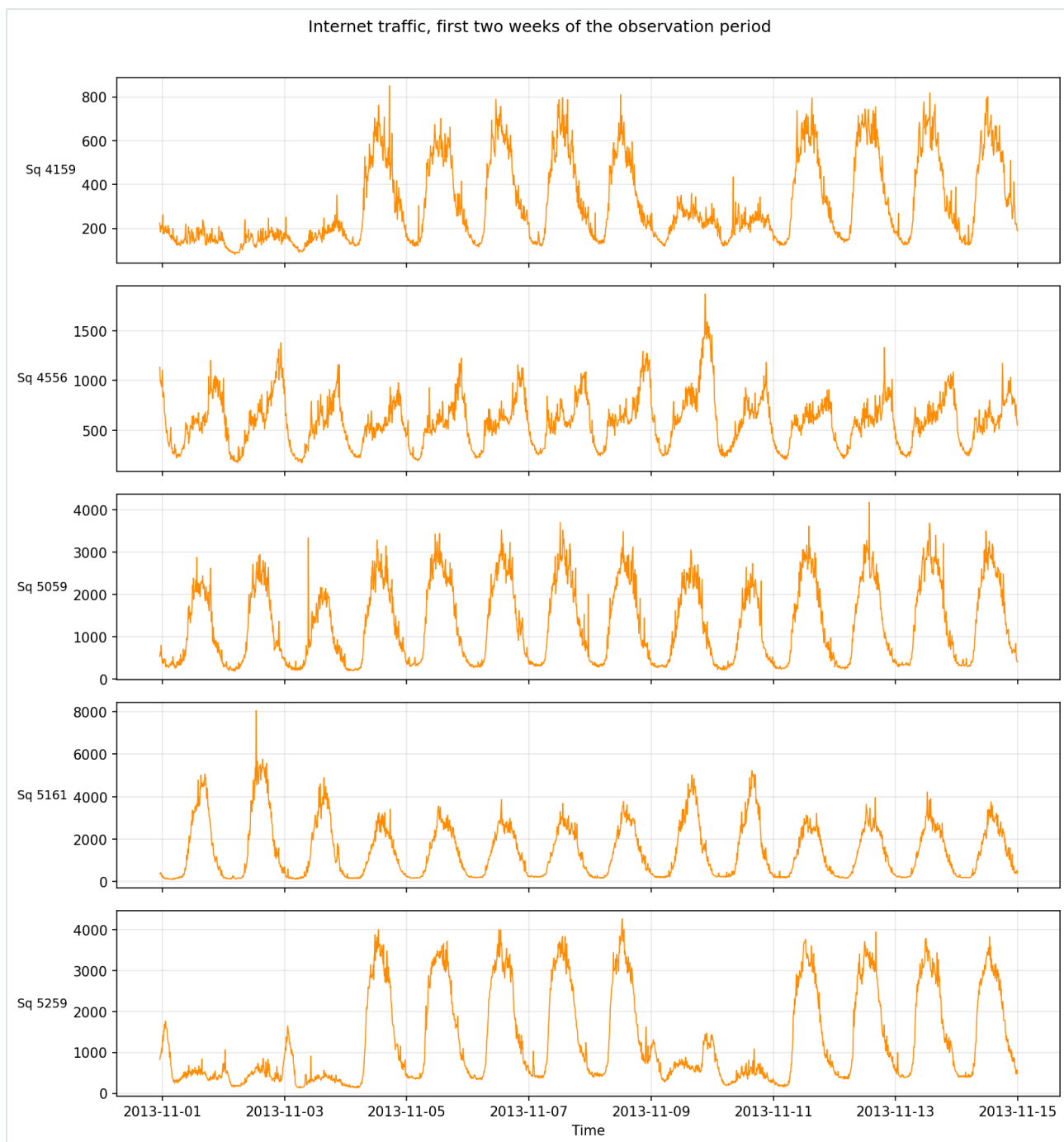
Figure 1. Distribution of total Internet traffic per square, full 2-month observation period (10,000 squares).



4.2 Top-3 squares and required squares

The three squares with the highest total Internet traffic over the full two-month observation period are **5161**, **5059**, and **5259** (totals: 12.68M, 11.09M, and 10.44M respectively — see `data/processed/target_squares.yaml`) — the same three squares identified when this ranking was first computed over a shorter, one-month provisional window, confirming the ranking is stable. Together with the two squares specified by the brief, 4159 and 4556, their first-two-weeks traffic is shown in Figure 2.

Figure 2. Internet traffic, first two weeks of the observation period, for the 3 highest-traffic squares plus the 2 brief-required squares.



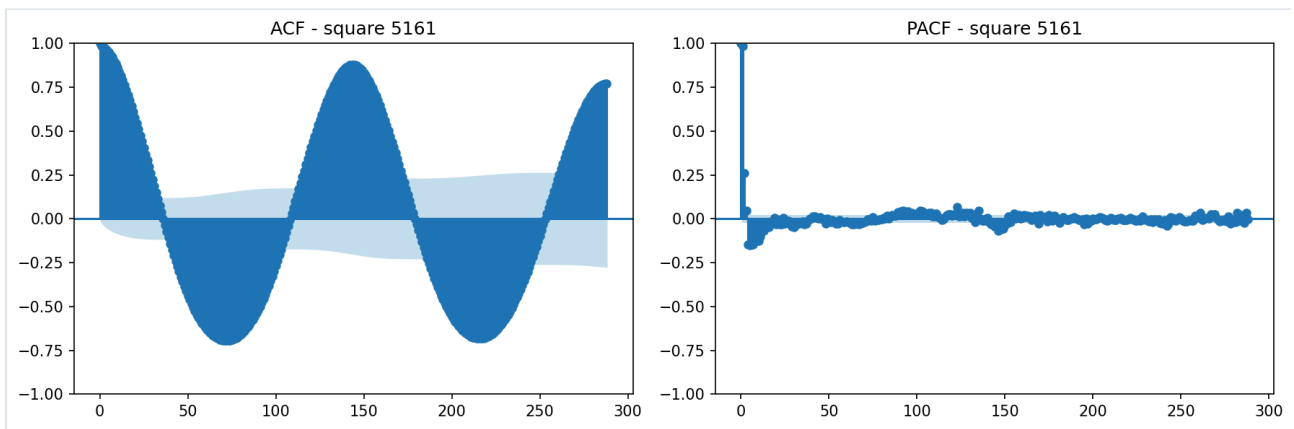
All five squares show a clear **daily commuting rhythm** — a trough around 04:00–06:00 and a broad daytime plateau/peak — but differ substantially in *scale*, *peak shape*, and *weekday/weekend contrast*: - **Square 4159** shows the strongest weekday/weekend contrast of the five: visibly lower and flatter peaks on the weekend (Nov 9–10), consistent with a business/office-dominated area where demand is commute-driven. - **Square 4556** has a comparatively "noisier", less sharply peaked profile with a higher baseline overnight, suggesting a more residential or mixed-use area with less pronounced day/night contrast. - **Squares 5059, 5161, and 5259** (the top-3) show the largest peaks and are visibly correlated with each other in timing, consistent with them being geographically adjacent, high-density squares. **Square 5161** is the most volatile of the three, showing two large, short-lived spikes (Nov 3 and Nov 10, both Sundays) that are 2–3× its typical Sunday peak — flagged

here as the square most likely to stress-test a model's ability to track sudden, non-repeating deviations, revisited directly in the failure-case analysis of Section 6.3.

4.3 Additional analyses on the highest-traffic square (5161)

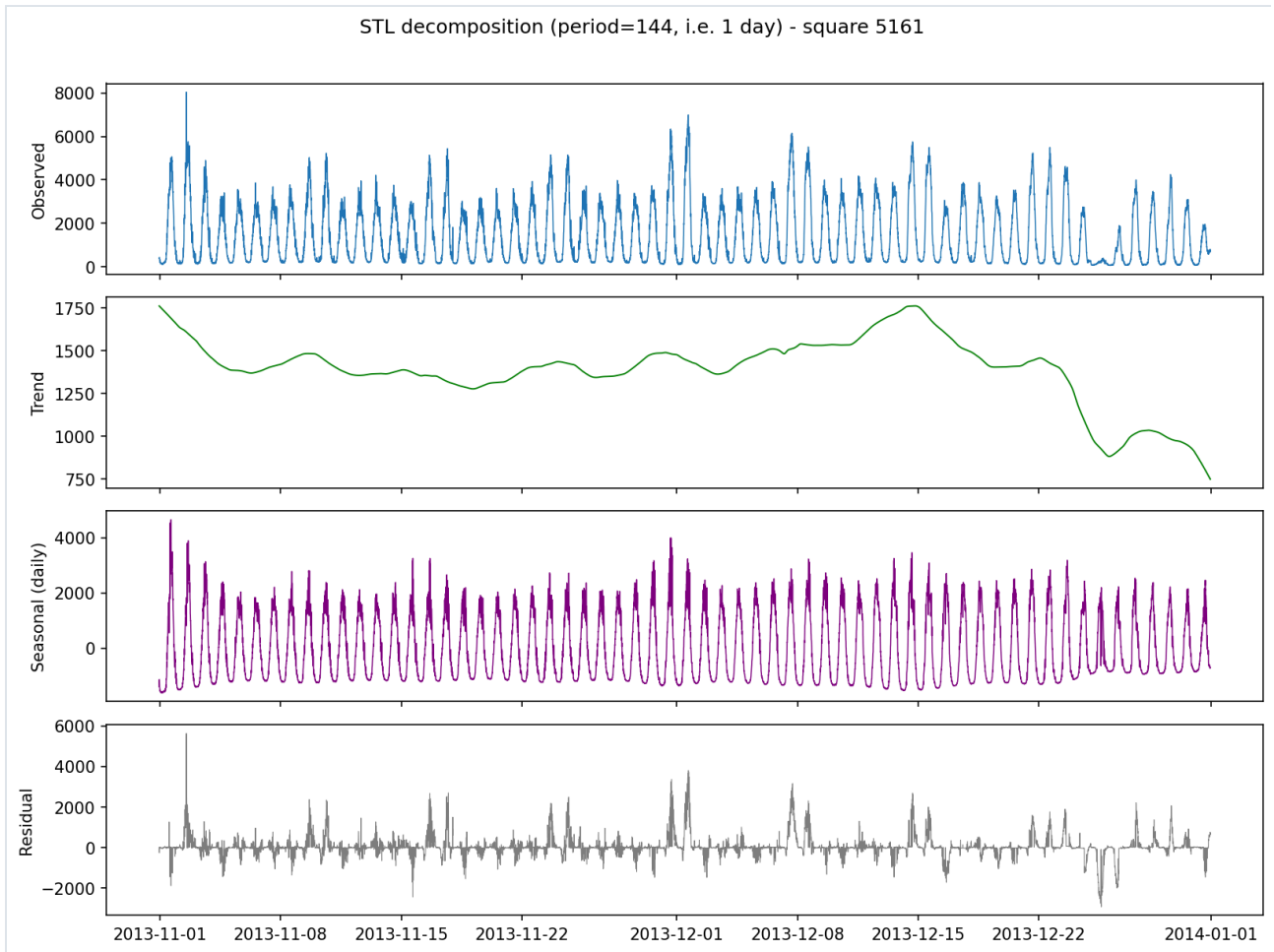
(a) Autocorrelation (ACF/PACF). Figure 3 shows a strong, slowly-decaying periodic pattern with peaks at lag ≈ 144 and ≈ 288 (i.e. 1 and 2 days at 10-minute resolution) and troughs at half-day offsets — a textbook signature of dominant daily seasonality. The PACF cuts off sharply after 1–2 lags, indicating that, once daily seasonality is accounted for, the remaining short-term dependence is low-order. **Implication for modelling:** this justifies (i) using a *low-order* autoregressive component for SARIMA combined with an explicit seasonal term rather than a large p , and (ii) a sequence length of at least 144 steps (1 day) for the LSTM/TCN so they can see a full seasonal cycle.

Figure 3. ACF and PACF, square 5161.



(b) STL decomposition and stationarity (ADF test). Figure 4 confirms a large, stable daily seasonal component and a slowly-varying trend, with residuals that are small most of the time but show sharp spikes aligned with the anomalous days noted above. The Augmented Dickey-Fuller test on the raw level series rejects the unit-root null (ADF statistic ≈ -19.13 , $p \approx 0$; also rejected on the first difference, statistic ≈ -15.05 , $p \approx 9.2 \times 10^{-28}$ — see `results/stationarity_tests.csv`). **This is a useful but partial signal:** ADF tests for a stochastic trend, not for seasonality, so a "stationary" verdict here does not mean the series lacks structure to model — it means differencing is *not* strictly required to remove a trend, but the strong deterministic-like daily cycle still needs to be modeled explicitly, which directly motivates the modelling choices in Section 5.

Figure 4. STL decomposition (period=144), square 5161.



5. Methodology

5.1 Forecasting task

Following the assignment's formalization, at each 10-minute interval t in the evaluation week, each model receives the history of `internet` traffic in area a up to and including t , and must produce a one-step-ahead estimate for $t+1$. This is evaluated in a **walk-forward** fashion across the entire evaluation week: predictions are made at every step using the *true* observed history up to that point (not the model's own previous predictions), matching the assignment's formal definition and giving a fair, comparable one-step error for every model.

5.2 Train / evaluation split

For each of the three target squares, the model is trained on **2013-11-01 00:00 → 2013-12-15 23:50** (45 days) and evaluated one-step-ahead across the assignment-specified week, **2013-12-16 00:00 → 2013-12-22 23:50** (7 days, 1,008 points).

5.3 Model-specific input representation, preprocessing, and training

SARIMA + Fourier terms. Input: the univariate `internet` series only (no external features). Unlike LSTM/TCN, SARIMA has no fixed input *sequence length* — its state-space representation

conditions each forecast on the entire growing training history via the Kalman filter's internal state, updated incrementally as each new true observation arrives during walk-forward evaluation, rather than on a truncated sliding window. Seasonality is represented with Fourier (sin/cos) regressors at the 144-step (1-day) period — chosen instead of a full seasonal-ARIMA term (`seasonal_order=(P,D,Q,144)`) because fitting the latter via maximum likelihood at a 144-step seasonal period is computationally impractical to fit and, especially, to tune; Fourier terms give an equivalent smooth seasonal representation at a small fraction of the cost [2, ch. 12] — see `common.py::fourier_terms`. (`p`, `d`, `q`) order and the number of harmonics are chosen by grid search (Section 5.4). No additional scaling is applied. At walk-forward time, the model's Kalman-filter state is updated with each newly observed true value via `.append(..., refit=False)` (no re-estimation of coefficients during the evaluation week). The SARIMAX wrapper enforces `enforce_stationarity=True`, `enforce_invertibility=True`, constraining the optimizer to AR/MA roots outside the unit circle; leaving both unconstrained (statsmodels' own example default) is known to let the optimizer converge to explosive AR coefficients on some squares, producing forecasts that diverge to $\pm\infty$ within a few walk-forward steps, so this safeguard is applied unconditionally rather than only when a problem is observed.

LSTM. Input: a sliding window of the last `sequence_length` scaled `internet` values (single feature). Values are scaled with a min-max scaler fit on the training window only (bounds derived only from training data, applied unchanged to the evaluation window, to avoid leakage). Architecture: two stacked LSTM layers with dropout, followed by a single dense output unit; trained with Adam. **Early stopping on a held-out validation slice is used both during hyperparameter tuning and in the final model reported in Section 6:** the last `VAL_DAYS=3` days of the training window are held out, the scaler is fit on the remaining (reduced) training portion only, and `EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True)` is active on that held-out slice for the final fit, exactly mirroring the tuning protocol rather than diverging from it. At evaluation time, one-step-ahead predictions use the *true* preceding values (teacher forcing) drawn from the full training history (not the reduced, validation-excluding portion used only for fitting), consistent with the walk-forward protocol.

TCN. Input: the same sliding window as LSTM (single feature, min-max scaled the same way, same validation-gated early-stopping protocol as LSTM — see above). Architecture: **6 residual blocks** of dilated causal 1D convolutions, dilation rate doubling per block (1, 2, 4, 8, 16, 32), kernel size 3, with dropout after each convolution; each block adds a 1×1 convolution projection on its shortcut connection only where the channel count changes (i.e. only in the first block, where the input's single channel is projected up to the block's filter count). This gives a receptive field of ≈ 253 steps, comfortably covering the 144-step window. The stack is followed by `GlobalAveragePooling1D` (averaging the learned representation across the whole window into a fixed-size vector) and a single dense output unit. Unlike LSTM's or SARIMA's explicit dependence on the *most recent* observation, this pooling step gives every position in the window equal weight by construction — a design property that turns out to matter for the results in Section 6.3.

5.4 Hyperparameter tuning strategy

Given three structurally different models, an iterative, purely manual search across all of them was impractical within the available time budget; **grid search** was used instead, run once on the single highest-traffic square (5161, most representative and highest signal-to-noise), with the last 3 days of the training window held out as a validation slice strictly *before* the evaluation week (so the evaluation week is never touched during tuning). Every combination tried is logged with its parameters, RMSE/MAE/MAPE, and wall-clock time (`results/tuning_results_sarima.csv`, `results/tuning_results_lstm_tcn.csv`). Grids:

- **SARIMA**: $(p,d,q) \in \{(1,1,1), (2,1,2), (3,1,1)\} \times \text{Fourier harmonics} \in \{0, 2, 4\}$.
- **LSTM**: sequence length $\in \{72, 144\} \times \text{units} \in \{[32], [64,32]\} \times \text{learning rate} \in \{0.001, 0.0005\}$ (15 tuning epochs with early stopping, patience 3; the winning configuration is retrained for up to 40 epochs, with early stopping still active, for the final Section 6 results).
- **TCN**: sequence length $\in \{72, 144\} \times \text{filters} \in \{16, 32\} \times \text{learning rate} \in \{0.001, 0.0005\}$, with the same 15-epoch/early-stopping protocol. Architecture (block count, dilation schedule, kernel size) was fixed by design rather than tuned, so this grid mirrors LSTM's shape (sequence length $\times$ capacity $\times$ learning rate) using `filters` as TCN's capacity knob in place of LSTM's `units`.

6. Results and Discussion

6.1 Hyperparameter tuning outcome

Grid search on square 5161 (Section 5.4) selected: **SARIMA** order `(2,1,2)` with **4** Fourier harmonic pairs; **LSTM** sequence length 72 with a single `[32]`-unit layer at learning rate 0.001; **TCN** sequence length 144 with 32 filters at learning rate 0.001. Two patterns are worth noting: (i) for SARIMA, adding Fourier harmonics steadily reduced validation RMSE ($\approx 184 \rightarrow \approx 154$ going from 0 to 4 harmonics at order `(1,1,1)`), confirming the daily seasonal component carries real predictive value, while the choice of `(p,d,q)` order made comparatively little difference ($\approx 153\text{--}154$ RMSE across all three orders at 4 harmonics) — consistent with the EDA's PACF finding of low-order residual dependence once seasonality is accounted for; (ii) for LSTM, the *simplest* configuration tried (`sequence_length=72`, a single 32-unit layer) won outright (RMSE ≈ 200.5 , versus $\approx 220\text{--}308$ for every deeper/longer alternative) — with more training data now available (45 days vs. the 23-day window used in an earlier, data-limited iteration of this pipeline) and validation-gated early stopping genuinely active, the smaller, faster-to-fit configuration generalized best rather than needing extra capacity.

6.2 Per-square performance

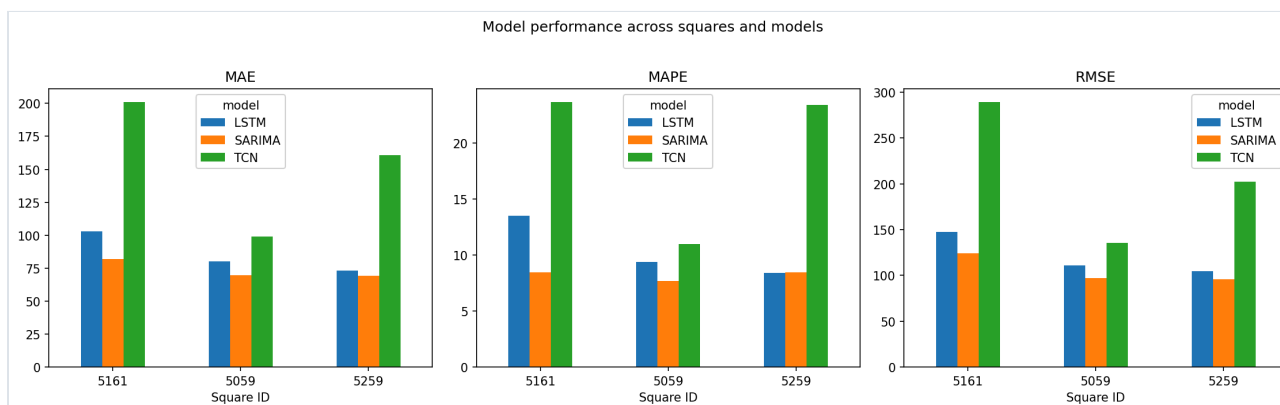
Timing methodology. All fit/predict times reported below were measured with `time.perf_counter()` wrapped directly around each model's `.fit()` call and around the full walk-forward prediction loop, on a single run per (model, square) pair — not averaged across repeated trials, so individual values carry some run-to-run variance (background OS load, thermal

throttling), though the relative ordering between models (discussed below) is large enough to be robust to that noise. All experiments, including these timings, were run on a **MacBook Air, Apple M2 chip (8 cores: 4 performance + 4 efficiency), 8 GB unified memory, macOS 26.7.1**, using Python 3.11.15 in a single-process, CPU-only environment (no GPU acceleration for the LSTM/TCN training) — the same 8 GB memory ceiling that motivated the ingestion strategy in Section 3.2.

Square	Model	MAE	MAPE (%)	RMSE
5161 (top-1)	SARIMA	81.91	8.44	123.84
5161	LSTM	103.04	13.52	147.58
5161	TCN	201.04	23.65	289.20
5059 (top-2)	SARIMA	69.73	7.69	96.75
5059	LSTM	80.34	9.37	110.78
5059	TCN	99.04	10.99	135.56
5259 (top-3)	SARIMA	69.15	8.46	95.94
5259	LSTM	73.16	8.38	104.49
5259	TCN	160.85	23.41	202.02

(Full tables: `results/model_performance_square_{5161,5059,5259}.csv` ; cross-square/model bar chart: Figure 5; composite actual-vs-predicted overlays for all 3 squares: Figure 6 and `figures/comparison_sq*.png` ; the full 9 individual model×square plots are in `figures/pred_sq*_.png` and render inline in `04_model_comparison.ipynb`.)

Figure 5. Model performance across squares and models.



SARIMA wins on MAE and RMSE on every square, the most consistent result of the three models. **LSTM is second on every square on MAE/RMSE, and clearly ahead of TCN on every square and every metric.** The one place the ranking is not clean-swept: **on square 5259, LSTM's MAPE (8.38) is marginally lower than SARIMA's (8.46)** — the only metric/square combination where SARIMA does not lead outright, though the two are close enough (a 0.08-point gap) that this is best read as a near-tie rather than a genuine reversal. **TCN is the clear weakest model on every square and every metric**, and the gap is large: on square 5161 — the square

flagged in the EDA as the most volatile of the three — TCN's RMSE (289.20) is more than double SARIMA's (123.84). Timing tells a more nuanced story than accuracy alone: with early stopping now genuinely active for both neural models (Section 5.3), **LSTM trains faster than TCN on every square** ($\approx 20\text{--}26$ s vs. TCN's $\approx 37\text{--}81$ s, e.g. 19.8 s vs. 37.4 s on square 5161 — see `results/timing_stats.csv`), a reversal from what a purely architectural comparison would predict (TCN's parallel convolutions are cheaper per epoch than LSTM's recurrence). The explanation is early stopping's patience-based criterion, not raw per-epoch cost: LSTM's validation loss plateaus and triggers a stop well before TCN's does, so LSTM runs fewer total epochs despite each epoch being relatively more expensive. Inference is near-instantaneous for both once trained (well under 1 ms/step). SARIMA trains fastest of all ($\approx 2\text{--}14$ s) but its walk-forward *inference* is by far the most expensive per step ($\approx 63\text{--}71$ ms/step, from the per-step Kalman-filter state update via `.append(refit=False)`) — two orders of magnitude slower per step than LSTM/TCN inference, though still fast enough in absolute terms (≈ 1 minute for a full 1,008-step evaluation week) not to be a practical concern here.

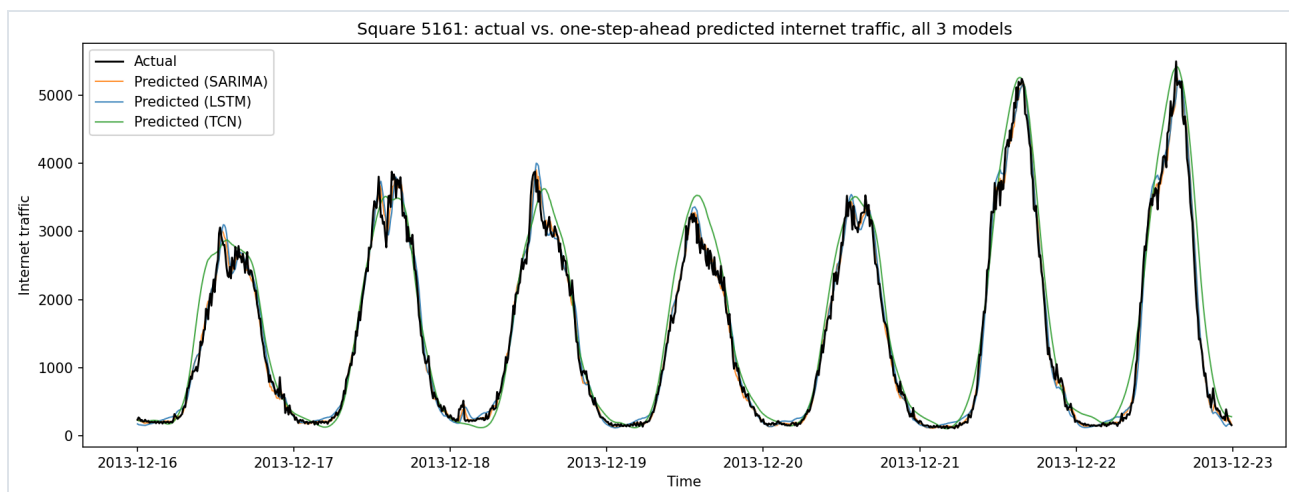
Why, given the data characteristics from Section 4: the EDA found a signal dominated by a strong, *stable, low-noise* daily cycle (ACF peaking at lag 144, PACF cutting off after 1-2 lags) with comparatively short low-order dependence once that cycle is accounted for. This is close to a best-case scenario for a linear model with an explicit seasonal term: SARIMA does not need to *discover* the seasonal pattern, it is handed the daily Fourier basis directly, leaving only a low-order AR/MA residual structure to estimate. LSTM, given 45 days of training history and a validation-gated stopping criterion that avoids overfitting, closes much of that gap — its simplest tuned configuration generalizes competitively, occasionally (square 5259's MAPE) matching SARIMA outright. TCN's much larger error is not explained by insufficient data in the same way: it saw the *same* 45 days of training history as LSTM and used the *same* validation-gated stopping criterion, yet remains far behind both other models on every square. This points to an architectural rather than a data-volume explanation, which Section 6.3 investigates directly.

Revisiting Section 2's LSTM justification against this result. LSTM was selected partly on the expectation that its recurrence would capture the bursty, non-repeating residual structure — visible in the STL residual plot — that a linear model cannot. If that advantage were showing up here, it should be most visible on square 5161, the square Section 4.2 identifies as the most volatile of the three (the anomalous Sunday spikes). It is not: LSTM's *closest* approach to SARIMA is instead on square 5259 — the lowest-traffic and comparatively calmest of the three top squares — where LSTM edges ahead on MAPE, while on the volatile square 5161 LSTM still trails SARIMA by a clear margin on every metric (RMSE 147.58 vs. 123.84, MAPE 13.52 vs. 8.44). The most likely explanation is that even 45 days of history is still not enough for LSTM to learn square 5161's specific, non-repeating anomalies (two isolated Sunday spikes in two months of data is a very small sample to generalize from) as anything other than noise — so the "bursty structure" advantage anticipated in Section 2 did not materialize where it was expected to matter most, at least at this data volume and this level of hyperparameter search. This is a genuine gap between the original justification and the observed result, not a case that is fully closed by these experiments.

6.3 Failure case: TCN's pooled representation under-tracks high-frequency structure

The clearest, most instructive failure is on **square 5161** — the square flagged in the EDA (Section 4.2) as the most volatile of the top-3, with two anomalous Sunday spikes during the observation period. TCN's RMSE here (289.20) is **133% higher** than SARIMA's (123.84), the largest relative gap of any square (versus 40% on square 5059 and 111% on square 5259). Figure 6 overlays all three models' predictions against the actual series for the full evaluation week and shows why: SARIMA and LSTM both track the actual signal closely, including its daily peaks and the sharper secondary peak on Dec 20, while TCN's forecast is visibly **smoother and phase-shifted** relative to the actual signal — it consistently **rounds off sharp peaks** (e.g. the Dec 16 and Dec 22 peaks, where actual traffic reaches ~5,000–5,500 but TCN's prediction lags and under- or overshoots the true peak timing) and **overshoots during troughs and shoulders** (visible around Dec 16–17 and Dec 19), producing what looks like a locally-averaged version of the true trajectory rather than a forecast that tracks its immediate fluctuations. The same pattern is visible on square 5259 (figures/comparison_sq5259.png), most dramatically during an irregular weekend dip around Dec 21–22, where TCN's prediction swings well above and then well below the actual (comparatively flat) signal while SARIMA and LSTM both track it closely. Square 5059 (figures/comparison_sq5059.png) shows the mildest version of the same pattern, consistent with it having the smallest SARIMA-vs-TCN relative gap of the three (40%, vs. 111% and 133% above): TCN tracks the daily cycle reasonably well for most of the week, with only minor peak-rounding, but still visibly diverges late in the week — undershooting sharply through the evening of Dec 22 into Dec 23 while SARIMA and LSTM continue tracking the actual signal. That the same smoothing/lagging behaviour appears at all three intensities in proportion to each square's relative TCN-vs-SARIMA gap is further evidence that this is a general property of the architecture on this task, not an artefact of any single square.

Figure 6. Square 5161: all 3 models' one-step-ahead predictions overlaid on the actual series, evaluation week.



A plausible structural explanation, consistent with the architecture described in Section 5.3: TCN's `GlobalAveragePooling1D` head compresses its entire 144-step receptive field into a single fixed-size vector, weighting every position in the window equally by construction. SARIMA, by contrast, explicitly conditions its next-step forecast on the *most recent* true observation at every walk-forward step (via the Kalman-filter state update), and LSTM's recurrent hidden state is updated sequentially, which naturally emphasises the most recent inputs it has seen. Neither of TCN's comparators has this

"equal-weighting" property, and — critically — this gap persisted even once TCN was given the same 45 days of training data and the same validation-gated early stopping as LSTM (Section 5.3), which rules out "TCN just needed more data" as the explanation and points squarely at the architecture itself.

Practical implication: for an operator choosing between these three models for a heterogeneous set of areas, TCN's architecture should not be deployed as-is on squares with pronounced volatility or anomalies without further changes (e.g. replacing or supplementing global average pooling with a mechanism that privileges the most recent timesteps, such as a final recurrent/attention layer or simply using the last-timestep feature vector instead of a pooled average) — this is a structural limitation, not one that more training data resolves on its own.

7. Conclusion and Future Work

This project investigated one-step-ahead Internet-traffic forecasting on the Milan CDR dataset over the assignment-specified evaluation week (December 16–22, 2013), comparing a linear/seasonal statistical baseline (SARIMA with Fourier terms), a recurrent deep learning model (LSTM), and a temporal convolutional network (TCN) across the three highest-traffic grid squares. Three findings stand out. First, **the model that best fits the data's actual characteristics led on the primary error metrics on every square:** SARIMA, given the daily seasonality identified during EDA directly as a Fourier basis, had the lowest MAE and RMSE on all three squares, though LSTM matched it on MAPE for the least volatile square — evidence that a well-specified linear model with explicit seasonality remains a strong baseline even against tuned neural alternatives. Second, **LSTM's recurrence closed much of the gap with SARIMA once given adequate training data and genuinely validation-gated training:** its simplest tuned configuration (a single 32-unit layer, 72-step window) generalized better than deeper alternatives, and it comfortably outperformed TCN on every square. Third, **TCN's much larger error was not a data problem:** given the identical training history and validation-gated stopping criterion as LSTM, it still trailed by a wide margin, and the failure mode documented in Section 6.3 — a smoothed, phase-shifted forecast that misses sharp peaks and troughs — is explainable directly from its `GlobalAveragePooling1D` head's lack of recency-weighting, a structural rather than data-driven limitation.

Margins for improvement, per model. SARIMA: a single fixed `(p,d,q)` order was chosen once via grid search and reused across all three squares; an automatic per-square order search (`auto_arima`-style) with a validity check, plus periodic re-fitting over a longer deployment horizon, would likely improve robustness without sacrificing its current accuracy. **LSTM:** already benefits from adequate data and validation-gated training in this iteration; the next lever is architecture search beyond the two-point grid tested here (e.g. attention over the input window) to test whether it can fully close the remaining gap with SARIMA rather than only approaching it. **TCN:** the clearest, most specific improvement identified by this report is architectural, not a matter of more data or training time (both were already controlled for) — replacing or supplementing `GlobalAveragePooling1D` with a mechanism that privileges recent timesteps (e.g. taking only the last timestep's features, or adding a small attention layer over the pooled sequence) directly targets

the failure mode demonstrated in Section 6.3, and would be the first thing to try before concluding TCN is unsuitable for this signal.

Limitations. (i) Hyperparameters were tuned on a single square (5161) and reused across all three for tractability; a per-square tuning pass might close some of the gap between models on squares where their ranking is close (e.g. square 5259's LSTM/SARIMA MAPE near-tie). (ii) TCN's architecture (block count, dilation schedule, kernel size) was fixed by design rather than searched; only its capacity (`filters`), input window, and learning rate were tuned, so a broader architecture search — particularly one that addresses the pooling-head limitation identified in Section 6.3 — might change the TCN-vs-LSTM comparison. (iii) SARIMA's numerical fragility (its optimizer can converge to explosive coefficients without the stationarity/invertibility safeguard applied here) means it needs that safeguard to be deployed safely across many areas unattended — a per-square automatic order-search with a validity check would be a more robust production approach than one fixed order. (iv) The walk-forward SARIMA evaluation updates the Kalman filter state with true observations every step but never re-estimates its coefficients; over a longer deployment horizon than one week, periodic re-fitting would likely be needed as the underlying pattern drifts. (v) The UTC/local-day boundary offset noted in Section 3.2 means the observation window used for ranking is shifted by roughly one hour from its nominal dates — small in practice, but worth fixing in a production system with a timezone-aware filter.

Future work. Extend the comparison to more than the top-3 squares, including low-traffic/residential squares where TCN's pooling-head weakness (Section 6.3) may be even more pronounced; modify the TCN head (e.g. take the last timestep's features instead of pooling, or add an attention layer) to test whether that closes the gap with LSTM and SARIMA; add cross-square (spatial) features, since neighbouring squares in the same transit corridor showed visibly correlated dynamics in the EDA; and evaluate a hybrid approach (e.g. SARIMA residuals modeled by a neural correction term) to combine the seasonal-baseline strength of SARIMA with nonlinear residual-fitting.

References

- [1] G. Barlacchi et al., "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," *Scientific Data*, vol. 2, p. 150055, 2015.
- [2] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. OTexts, 2018.
- [3] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. John Wiley & Sons, 2015.
- [4] A. Azari, P. Papapetrou, S. Denic, and G. Peters, "Cellular Traffic Prediction and Classification: A Comparative Evaluation of LSTM and ARIMA," in *Discovery Science (DS 2019)*, LNCS vol. 11828, Springer, 2019, pp. 129–144.
- [5] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

- [6] G. L. Santos, P. Rosati, T. Lynn, J. Kelner, D. Sadok, and P. T. Endo, "Predicting short-term mobile Internet traffic from Internet activity using recurrent neural networks," *International Journal of Network Management*, 2021.
- [7] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, 2018.
- [8] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conference*, 2010, pp. 56–61.

Full BibTeX entries: `report/references.bib`.

GitHub repository: https://github.com/Manzi453/Formative1_Techinique1

Demo video: *[add link once recorded — see `report/video_script.md` for the presentation outline]*